PRACTICAL - 06

Aim: Remote control (Switchable.java + Driver.java)

Code Screenshot:

Switchable.java

```java
public interface Switchable {
    void on();
    void off();
    default void toggle() {
        if (isOn()) off(); else on();
    }
    boolean isOn();
}

@FunctionalInterface
interface SwitchCondition {
    boolean canSwitch(Switchable device, int hour);
}

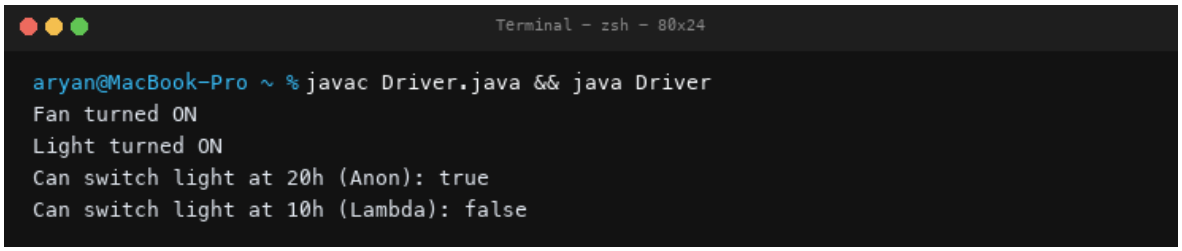
class Fan implements Switchable {
    private boolean state = false;
    public void on() { state = true; System.out.println("Fan turned ON"); }
    public void off() { state = false; System.out.println("Fan turned OFF"); }
    public boolean isOn() { return state; }
}

class Light implements Switchable {
    private boolean state = false;
    public void on() { state = true; System.out.println("Light turned ON"); }
    public void off() { state = false; System.out.println("Light turned OFF"); }
    public boolean isOn() { return state; }
}
```

Driver.java

```java
public class Driver {
    public static void main(String[] args) {
        Switchable[] devices = { new Fan(), new Light() };
        for (Switchable d : devices) d.toggle();
        SwitchCondition anonCond = new SwitchCondition() {
            public boolean canSwitch(Switchable d, int h) { return h >= 6 && h <= 22; }
        };
        SwitchCondition lambdaCond = (d, h) -> h >= 18;
        System.out.println("Can switch light at 20h (Anon):" + anonCond.canSwitch(devices[1], 20));
        System.out.println("Can switch light at 10h (Lambda):" + lambdaCond.canSwitch(devices[1], 10));
    }
}
```

Output Screenshot:



Learning Outcome:

- Learned interface definition, contract adherence, and default methods.
- Understood functional interfaces (@FunctionalInterface) in Java.
- Mastered the transition from anonymous inner classes to concise lambda expressions.
- Implemented multiple inheritance of type through interfaces.
- Understood default method evolution and method dispatch rules.

Aim: Notification senders (Notifier.java + Driver.java)

Code Screenshot:

Notifier.java

```java
@FunctionalInterface
public interface Notifier {
    void send(String msg);
}

interface Urgent {}

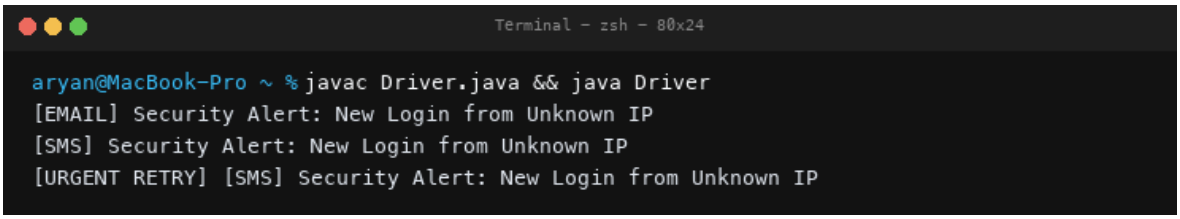
class EmailNotifier implements Notifier {
    public void send(String msg) { System.out.println("[EMAIL] " + msg); }
}

class SmsNotifier implements Notifier, Urgent {
    public void send(String msg) { System.out.println("[SMS] " + msg); }
}
```

Driver.java

```java
public class Driver {
    public static void main(String[] args) {
        Notifier email = new EmailNotifier();
        Notifier sms = new SmsNotifier();
        Notifier[] notifiers = { email, sms };
        String alert = "Security Alert: New Login from Unknown IP";
        for (Notifier n : notifiers) {
            n.send(alert);
            if (n instanceof Urgent) {
                System.out.print("[URGENT RETRY] ");
                n.send(alert);
            }
        }
    }
}
```

Output Screenshot:



Learning Outcome:

- Mastered functional programming concepts in Java using lambda expressions.
- Understood marker interfaces and how they provide metadata at runtime.
- Learned how to treat behaviors as first-class citizens in arrays.
- Implemented broadcast notification patterns with conditional dispatch.
- Practiced clean separation between interface definitions and caller logic.